

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char cmd[100];

    printf("Enter Linux command: ");
    scanf("%s", cmd);
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child PID : %d\n", getpid());
        execlp(cmd, cmd, NULL);
        perror("Execution Failed");
    }
    else if(pid > 0)
    {
        printf("Parent PID : %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }
    else
    {
        printf("Fork Failed\n");
    }
    return 0;
}
```

```

hemasai@Hemasai: ~
hemasai@Hemasai:~$ nano task1.c
hemasai@Hemasai:~$ gcc task1.c -otask1
hemasai@Hemasai:~$ ./task1
Enter Linux command: uname -a
Parent PID : 652
Child PID : 653
Linux
Child process completed.
hemasai@Hemasai:~$

```

```

Child process completed.
hemasai@Hemasai:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:   0-11
Vendor ID:              GenuineIntel
Model name:             12th Gen Intel(R) Core(TM) i5-1235U
CPU family:             6
Model:                 154
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):              1
Stepping:               4
BogoMIPS:               4992.00
Flags:                  fpu vme de pse tsc msr pae mce cx8
                        apic sep mtrr pge mca cmov pat pse3
                        6 clflush max fsr sse sse2 ss ht s
                        yscall nx pdpe1gb rdtscp lm constan
                        t-tsc rep_good nopl xtopology tsc_r
                        eliable nonstop_tsc cpuid tsc_known
                        _freq pni pclmulqdq vmx sse3 fma c
                        x16 pcid sseu_1 sse4_2 x2apic movbe
                        popcnt tsc_deadline_timer aes xsave
                        e avx f16c rdrand hypervisor lahf_l
                        m abm 3dnowprefetch ssbd ibrs ibpb
                        stibp ibrs_enhanced tpr_shadow ept
                        vpid ept_ad fsgsbase tsc_adjust bmi
                        1 avx2 smep bmi2 erms invpcid rdsee
                        d adx swap clflushopt clwb sha_ni v
                        saveopt xsavec xgetbv1 xsave_v
                        nni vmmi umip waitpkg gfni vaes vpc
                        lmulqdq rdpid movdiri movdir64b fsr
                        m rd_clear serial_ltr ibt flush_lid
                        arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
  L1d:                   288 MiB (6 instances)
  L1i:                   192 MiB (6 instances)
  L2:                    7.5 MiB (6 instances)
  L3:                    12 MiB (1 instance)
NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):     0-11
Vulnerabilities:
Gather data sampling:   Not affected
Ghostwrite:             Not affected
Indirect target selection: Not affected
Itlb multihit:         Not affected
L1tf:                   Not affected
Mds:                    Not affected
Meltdown:               Not affected
Mmio stale data:       Not affected
Old microcode:          Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed:               Mitigation; Enhanced IBRS
Spec rstack overflow:  Not affected
Spec store bypass:     Mitigation; Speculative Store Bypass
                        s disabled via prctl
Spectre v1:             Mitigation; usercopy/swapgs barrier
                        s and user pointer sanitization
Spectre v2:             Mitigation; Enhanced / Automatic IB
                        RS; IBPB conditional; PBRSSB-eIBRS S
                        W sequence; BMI BMI_D1S_5
Srbds:                  Not affected
Tsa:                    Not affected
Tsx async abort:       Not affected
Vsyscall:               Not affected
hemasai@Hemasai:~$

```

```

> ^C
hemasai@Hemasai:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M  1 disk
sdb   8:16   0 159.5M  1 disk
sdc   8:32   0     2G  0 disk [SWAP]
sdd   8:48   0     1T  0 disk /mnt/wslg/distro
/

hemasai@Hemasai:~$

```

```

hemasai@Hemasai:~$ ps -e
PID TTY          TIME CMD
  1 ?            00:00:01 systemd
  2 hvc0         00:00:00 init-systemd(Ub
  6 hvc0         00:00:00 init
 49 ?            00:00:00 systemd-journal
 81 ?            00:00:00 systemd-resolve
 90 ?            00:00:00 systemd-udev
150 ?            00:00:00 chronyd-starter
151 ?            00:00:00 cron
152 ?            00:00:00 dbus-daemon
156 ?            00:00:00 networkd-dispat
164 ?            00:00:00 systemd-logind
198 tty1         00:00:00 agetty
199 ?            00:00:00 rsyslogd
228 ?            00:00:00 chronyd
245 ?            00:00:00 chronyd
255 ?            00:00:00 unattended-upgr
344 ?            00:00:00 login
386 ?            00:00:00 systemd
388 ?            00:00:00 (sd-pam)
419 pts/1        00:00:00 bash
588 ?            00:00:00 SessionLeader
590 ?            00:00:00 Relay(593)
593 pts/0        00:00:00 bash
711 pts/0        00:00:00 ps
hemasai@Hemasai:~$

```

```

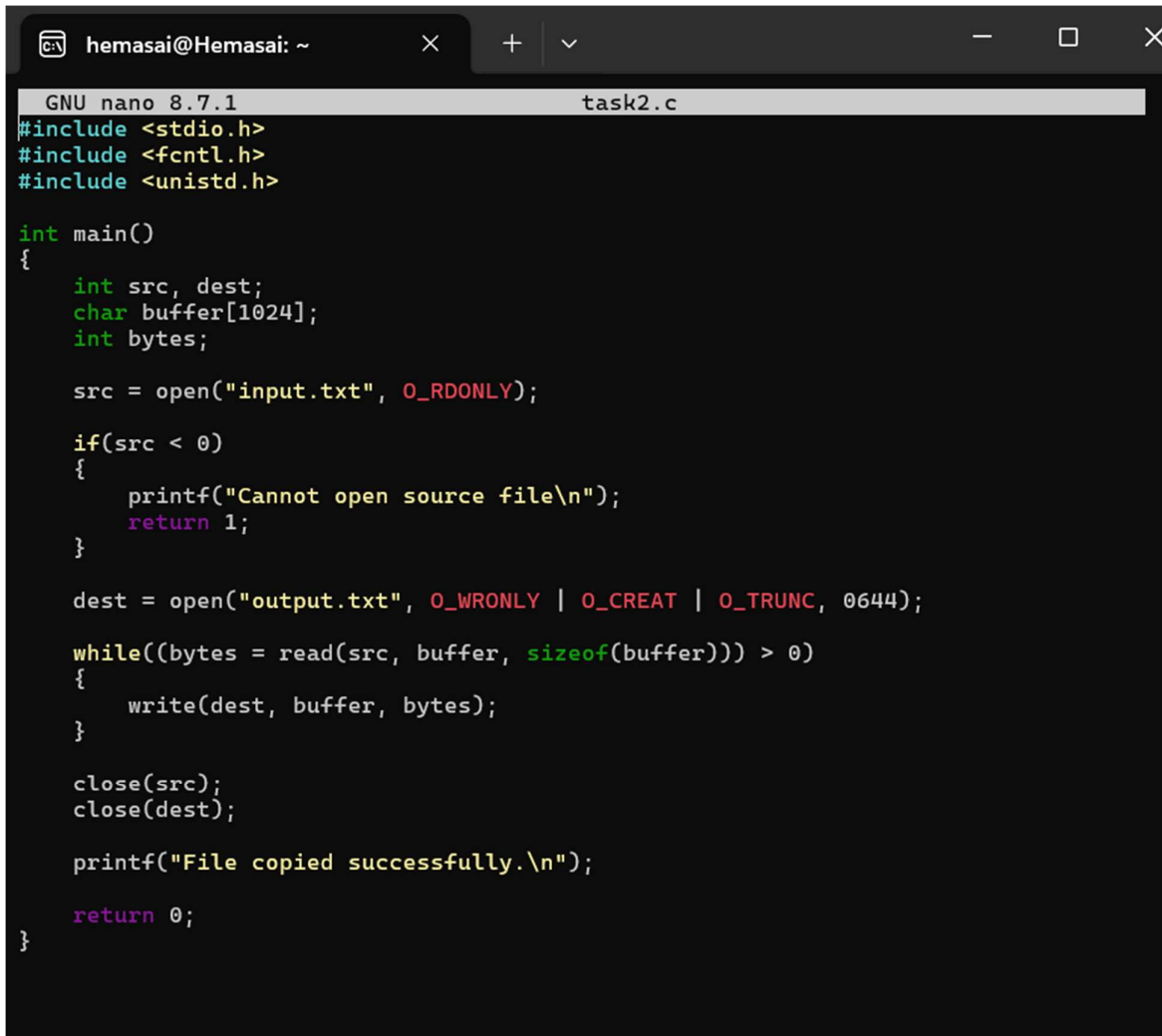
711 pts/0        00:00:00 ps
hemasai@Hemasai:~$ top
top - 09:26:06 up 12 min, 1 user, load average: 0.03, 0.06, 0.01
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.9 id,  0.1 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7776.6 total, 7273.3 free, 524.4 used, 123.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used. 7252.2 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   24212   15344  11564 S   0.0   0.2   0:01.17 systemd
    2 root        20   0    3180    2208   2072 S   0.0   0.0   0:00.03 init-syst+
    6 root        20   0    3180    2044   1940 S   0.0   0.0   0:00.00 init
   49 root       19  -1   50400   16816  15528 S   0.0   0.2   0:00.33 systemd-j+
   81 systemd+    20   0   22540   14600  11980 S   0.0   0.2   0:00.12 systemd-r+
   90 root       20   0   35360   12312   9228 S   0.0   0.2   0:00.50 systemd-u+
  150 root       20   0    2888    1948   1820 S   0.0   0.0   0:00.08 chronyd-s+
  151 root       20   0    4472    3140   2896 S   0.0   0.0   0:00.01 cron
  152 message+   20   0    8824    5384   4700 S   0.0   0.1   0:00.13 dbus-daem+
  156 root       20   0   44092   29692  14552 S   0.0   0.4   0:00.55 networkd-+
  164 root       20   0   18432    9440   8196 S   0.0   0.1   0:00.25 systemd-l+
  198 root       20   0    5492    2768   2580 S   0.0   0.0   0:00.01 agetty
  199 syslog     20   0  220548    5936   4656 S   0.0   0.1   0:00.16 rsyslogd
  228 _chrony     20   0   20516    7852   7000 S   0.0   0.1   0:00.10 chronyd
  245 _chrony     20   0   12092    2404   1760 S   0.0   0.0   0:00.00 chronyd
  255 root       20   0  123456   32648  18044 S   0.0   0.4   0:00.19 unattende+
  344 root       20   0    8648    5628   4784 S   0.0   0.1   0:00.02 login
  386 hemasai    20   0   22336   13588  10968 S   0.0   0.2   0:00.14 systemd
  388 hemasai    20   0   22908    4088   2088 S   0.0   0.1   0:00.00 (sd-pam)
  419 hemasai    20   0    6136    5476   3928 S   0.0   0.1   0:00.02 bash
  588 root       20   0    3196    1124    980 S   0.0   0.0   0:00.00 SessionLe+
  590 root       20   0    3196    1256   1108 S   0.0   0.0   0:00.03 Relay(593)
  593 hemasai    20   0    6268    5524   3848 S   0.0   0.1   0:00.11 bash
  723 hemasai    20   0    8184    6016   3900 R   0.0   0.1   0:00.01 top

```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.



```
hemasai@Hemasai: ~  
GNU nano 8.7.1 task2.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
int main()  
{  
    int src, dest;  
    char buffer[1024];  
    int bytes;  
  
    src = open("input.txt", O_RDONLY);  
  
    if(src < 0)  
    {  
        printf("Cannot open source file\n");  
        return 1;  
    }  
  
    dest = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
  
    while((bytes = read(src, buffer, sizeof(buffer))) > 0)  
    {  
        write(dest, buffer, bytes);  
    }  
  
    close(src);  
    close(dest);  
  
    printf("File copied successfully.\n");  
  
    return 0;  
}
```

